

Equivariant Graph Neural Networks (EGNN) in Loma

CHUHAO CHEN, University of California San Diego
ZHONGMING YU, University of California San Diego

1 INTRODUCTION

As an important subdomain in deep learning, the Equivariant Neural Network is widely used for data like point clouds, graphs, etc. Our proposal will be based on the Equivariant Graph Neural Network (EGNN) [3], which has proved to be a more powerful Equivariant Neural Network than the Tensor Field network [4] mentioned in the project document. Our implementation is released at https://github.com/CzzzzH/EGNN_loma.

2 IMPLEMENTATION

2.1 Network Architecture

In this section, we will present the structure of Equivariant Graph Neural Networks (EGNNs). We consider a graph $G = (\mathcal{V}, \mathcal{E})$ with nodes $v_i \in \mathcal{V}$ and edges $e_{ij} \in \mathcal{E}$. In addition to the feature node embeddings $h_i \in R^{n_f}$ we now also consider an n -dimensional coordinate $x_i \in R^n$ associated with each of the graph nodes. The EGNN model will preserve equivariance to rotations and translations on these set of coordinates x_i and it will also preserve equivariance to permutation. Now let's talk about Equivariant Graph Convolutional Layer (EGCL). For the input, it will take the node embeddings h^l , coordinate embeddings x^l , and edge information $\mathcal{E} = (e_{ij})$. This transformation of EGCL will output h^{l+1} and x^{l+1} . The detailed equations are defined below:

$$\mathbf{m}_{ij} = \phi_e(\mathbf{h}_i^l, \mathbf{h}_j^l, \|x_i^l - x_j^l\|^2, a_{ij}) \quad (1)$$

$$x^{l+1} = x_i^l + C \sum_{j \neq i} (x_i^l - x_j^l) \phi_x(\mathbf{m}_{ij}) \quad (2)$$

$$\mathbf{m}_i = \sum_{j \neq i} \mathbf{m}_{ij} \quad (3)$$

$$\mathbf{h}_{l+1} = \phi_h(\mathbf{h}_i^l, \mathbf{m}_i) \quad (4)$$

where ϕ_e , ϕ_h and ϕ_x are edge operator, node operator, and coordinate operator respectively. These operators are all approximated by Multilayer Perceptrons (MLPs) in EGCL. $\mathbf{m}_i$ and $\mathbf{m}_{ij}$ are intermediate embeddings for nodes and edges. $C = \frac{1}{M-1}$ is a constant coefficient for taking the average.

2.2 Loma Operators

2.2.1 Design Architecture. To clearly schedule our code, we draw a mindmap for the Loma operator design as shown in Fig. 1. 6 different classes of operators in total are supported in our project. We implemented **Add()**, **Sub()**, **Mul()**, **Div()** from Binary class. To be specific, we further introduce a broadcast binary in order to support the status of different shapes between 1st and 2nd arguments. **Sqrt()**, **ReLU()**, **SiLU()**, **Sigmoid()** have implemented under the Unary class, while **Sum()** is implemented in Reduce class. Apart from the above operators, we also need to deal with three other

special operators. **Linear()** is a *Matmul* operator, which has a different argument list and complicated parallel strategy. For the Aggr part, we implemented the **SumAggr()** operator which represents the Eq. (3). For the loss class, we have the most commonly used **MSELoss()** and **MAELoss()**. These operators are all implemented in SIMD, either in ISPC or in OpenCL.

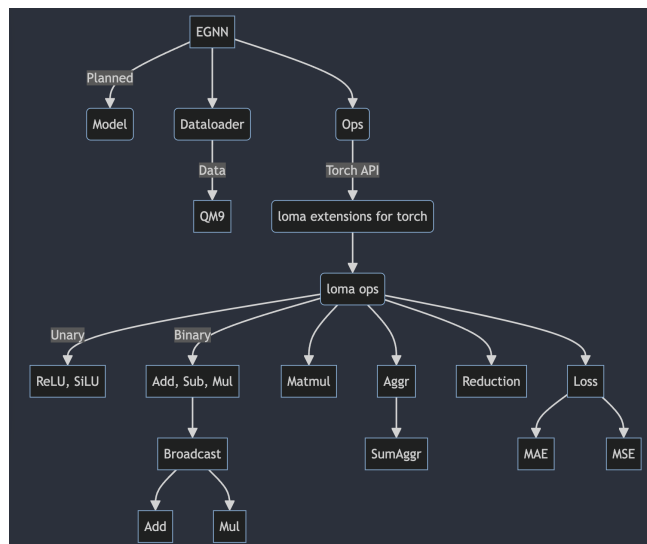


Fig. 1. Software Arch for EGNN Loma.

2.2.2 Operator List. To have a better understanding of our operators, we list the type inference (shape), parallel info, and atomics usage as shown in Table 1 (See [egnn/ops/loma/loma_code/ops.py](#)). Please note that due the limitation of SIMD codegen, we have to use the 1D parallelization method. That means there is only one loop to be unrolled in the final codegen stage. Thus, in most cases, we flatten the 2D operators into 1D and apply the elementwise strategy, such as **Binary**, **Unary**, etc. For **Matmul**, we have to use a blockwise strategy, where each block is limited to one output. Then we do the vector add sequentially within one block. Note that the 2nd argument for **Aggr** is indicated by the edge index of the given graph, which is an int array without autodiff.

2.3 QM9 Dataloader

2.3.1 Introduction. The QM9 dataset is a comprehensive collection of quantum chemical properties for a large number of molecules. This dataset has been widely used in the field of computational chemistry to develop and test machine-learning models that predict molecular properties. QM9 includes data for nearly 134,000 stable small organic molecules made up of C, H, O, N, and F. Each molecule consists of up to 9 non-hydrogen atoms.

Description	Shape	Parallel	Atomics
Binary	2D, 2D $\rightarrow$ 2D	Elementwise	No
BinaryBroadcast	2D, 1D $\rightarrow$ 2D	Elementwise	No
Unary	2D $\rightarrow$ 2D	Elementwise	No
Reduce	2D $\rightarrow$ 1D	Elementwise	Yes
Loss	2D, 2D $\rightarrow$ scalar	Elementwise	Yes
Aggr	2D, 1D $\rightarrow$ 2D	Elementwise	Yes
Matmul	2D, 2D $\rightarrow$ 2D	Blockwise	No

Table 1. Operator Description List.

2.3.2 Target Properties. The dataset includes several target properties that are relevant to different aspects of molecular chemistry. Table 2 is a selected overview of the targets we used from the QM9 dataset:

2.3.3 Code Description. In our repo, we achieve the data loader in three typical steps. (See [egnn/dataloader/qm9.py](#)). First of all, we will do **Transformation**. The Complete class implements a transformation that fully connects the graph representation of each molecule, ensuring that all possible edges between nodes are included. Secondly, we will do **Normalization**. Inside the dataQM9 class, we normalize the target properties (such as molecular energies or electronic properties) to have zero mean and unit variance. This standardization is important for the testing stage. Lastly, we will do the **Split**. The script automatically splits the dataset into training, validation, and test subsets. It then wraps these subsets in PyG’s [1] DataLoader, which provides batch-wise iteration over the datasets.

Property	Description	Unit
μ	Dipole moment	Debye
$\Delta\epsilon$	Gap between orbital energy	eV
G	Free energy at 298.15K	eV

Table 2. Target properties in the QM9 dataset.

3 EXPERIMENTS

We conducted several experiments to evaluate the correctness and efficiency of our Loma network. We also evaluate the effectiveness of using our EGNN on the QM9 prediction tasks by comparing it with the GCN network as a baseline.

3.1 Correctness Verification

For each Loma operator, we add a unit test to check the correctness of the forward and backward results. We use the corresponding PyTorch operator as the reference (See [egnn/ops/test_loma_ops.py](#)). We run the unit test at each compiled mode (C/ISPC/OpenCL) to guarantee that the Loma codes have the same behavior at different backend.

To verify the correctness of the whole EGNN implemented in Loma, we train both the EGNN in Loma and the EGNN in PyTorch for 20 epochs and compare their results. We plot the loss value

per train iteration and the validation MAE per epoch in Fig. 2 and Fig. 3. Note that we run the two networks on GPU under the same random seed, but the results are not totally the same. The difference is probably caused by the different floating point errors in different parallelization modes.

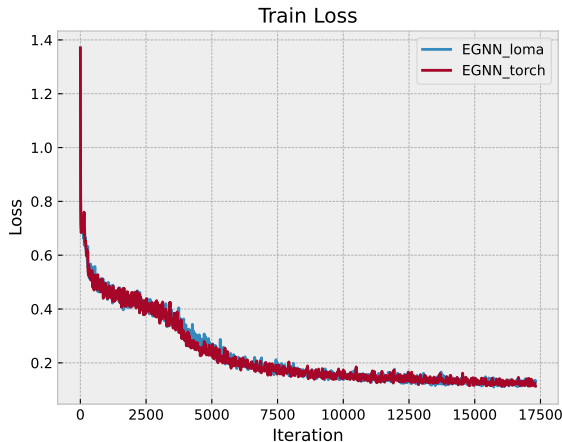


Fig. 2. Train Loss for EGNN in Loma and the EGNN in PyTorch

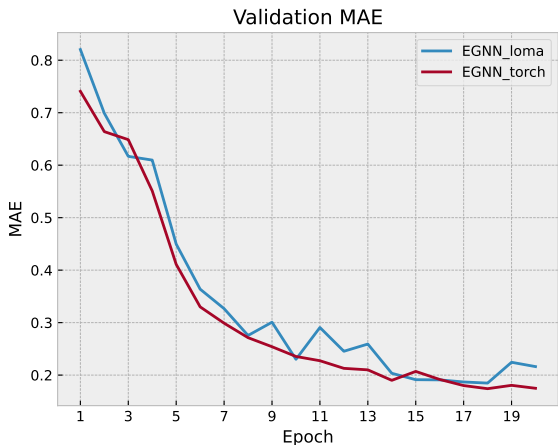


Fig. 3. Validation MAE for EGNN in Loma and the EGNN in PyTorch

3.2 Performance

We evaluate the performance of Loma on two neural network models: Simple MLP and EGNN (See [egnn/models/MLP_loma.py](#) and [egnn/models/EGNN_loma.py](#)). The two models are designed for MNIST task and QM9 task respectively. We present the quantitative in Table 3, where there are many interesting observations.

We observed that the performance bottleneck is at the linear layers for both models, where many matrix multiplications are applied. Indeed, the feature dimension of the linear layers in Simple MLP is the same order of magnitude as that of the linear layers

in EGNN (the former is around 512 and the latter is around 128). However, the batch size of the input in EGNN is much larger than that in Simple MLP (the former is around 40000 while the latter is only 64). Although our implementation parallelizes the calculation in the batch size axis, this huge gap in batch size makes our Loma linear layers much slower for EGNN in terms of running on the GPU. To be more specific, the inference time of Loma (OpenCL) for Simple MLP is comparable with that of PyTorch (CUDA), which is highly optimized, but the Loma (OpenCL) becomes 10 times slower than PyTorch (CUDA) when the model is changed to EGNN. This gap will be further magnified if we add the more complex backward process.

Another interesting observation is about the ISPC results. Since our Loma linear layers will launch a large number of threads in the calculation, the ISPC parallelization is expected to achieve minor or no improvement compared with the naive serial implementation. This conjecture is essentially correct in the training process, but we found that ISPC is super fast when it comes to inference. The performance of Loma (ISPC) is even better than Loma (OpenCL) for EGNN.

All the performance tests are run on AMD Ryzen 7 7800X3D CPU and Nvidia GeForce RTX 4090 GPU.

3.3 EGNN vs GCN

To evaluate the effectiveness of EGNN for QM9 prediction task. We compare it with Graph Convolutional Networks (GCN) [2], which do not ensure equivariance. We keep the compared GCN the same architecture as the EGNN, except for replacing the 4 EGCL with 4 Graph Convolutional Layers (GCL), which makes their scale similar.

3.3.1 Train Settings. We trained 3 EGNN models with different target properties described in Section 2.3.2. Each model is trained for 300 epochs, which takes around 3 hours for a Nvidia GeForce RTX 4090 GPU. We use ReduceLROnPlateau scheduler provided by PyTorch with an initial learning rate of 10^{-3} . The optimizer is the Adam with the default setting. We train GNN models using the same settings. Since the Loma models are too slow, we train the models using the PyTorch version. We have verified the correctness in Section 3.1, so there is no need to spend more time training Loma models.

3.3.2 Results. We report the average Mean Absolute Error (MAE) on the test set for EGNN and GCN in Table 4. The number is obtained from the best test epoch. Similar to Section 3.1, we plot the validation MAE for 3 different target properties in Fig. 4 Fig. 5 and Fig. 6. We can see that the Validation MAEs of GCN barely decrease for μ and $\Delta\epsilon$ during training, while those of EGNN decrease a lot. For G , the GCN does work a lot, but it is because the initial prediction is too inaccurate, and the final result is still much worse than that of EGNN.

Our prediction accuracy for EGNN didn't reach what was reported in the original EGNN paper. This is because (1) we simplified the original EGNN by reducing the number of layers and removing the extra attention modules and (2) we didn't train enough epochs to make the optimization converge. In this experiment, we aim to show

the effectiveness of EGNN but not to align our EGNN model with that in the original paper.

Target Property Units	μ D	$\Delta\epsilon$ eV	G eV
GCN	0.873	0.722	3.433
EGNN	0.081	0.099	0.617

Table 4. **Effectiveness Evaluation.** We compare the EGNN with GCN on 3 different target properties μ , $\Delta\epsilon$ and G .

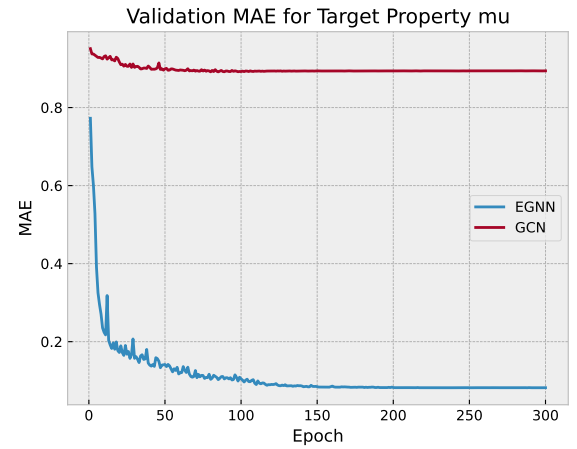


Fig. 4. Validation MAE for Target Property μ

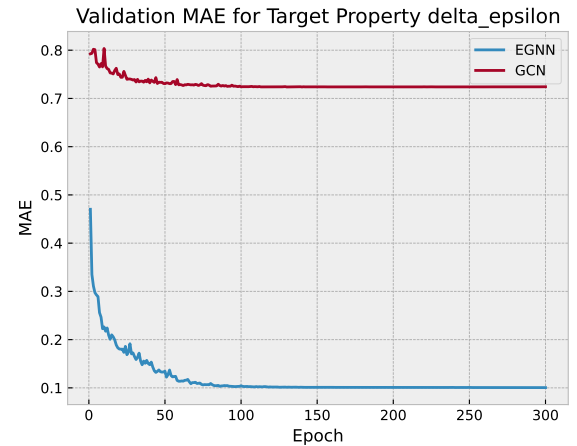


Fig. 5. Validation MAE for Target Property $\Delta\epsilon$

Model / Backend	Loma (C)	Loma (ISPC)	Loma (OpenCL)	PyTorch (CPU)	PyTorch (CUDA)
Simple MLP (train)	9.376	8.942	1.368	0.533	0.447
Simple MLP (inference)	2.786	0.707	0.513	0.377	0.328
EGNN (train)	787.9	379.9	148.0	13.39	5.356
EGNN (inference)	86.20	24.6	46.76	8.273	4.564

Table 3. **Performance Test.** We compare the train and inference performance of Loma models with PyTorch models on different backends. The number in the table means the time cost (second) per 100 iterations

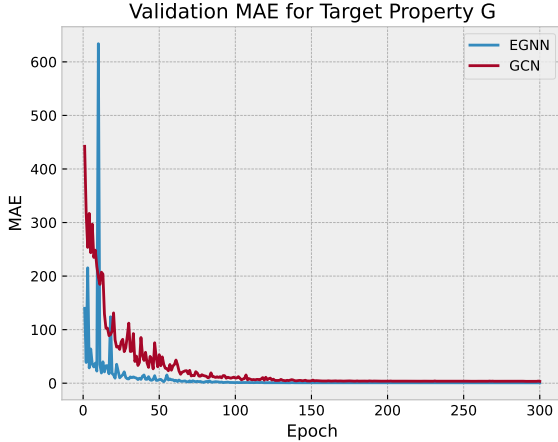


Fig. 6. Validation MAE for Target Property G

4 LIMITATION

4.1 Performance

As discussed in Section 3.2, the performance of loma operators drops fast as the input scale increases, which makes it impossible to train large neural networks. It is unlikely that the performance will improve much by merely modifying the SIMD loma code with the current compiler. So the next step to make loma neural networks more practical is to improve the compiler and also incorporate CUDA instead of OpenCL.

4.2 Memory Issue

PyTorch provides users with sophisticated memory management so we don't need to manually manage the GPU memory when using PyTorch operators to implement a neural network. Unfortunately, we have to care much about the memory issue when using Loma operators. The GPU buffers are created explicitly in our OpenCL implementation, and we need to release them. In our current implementation, we trace each GPU buffer and release all of them at the end of one train or inference step. This might not be the best strategy and it still needs to be improved.

Memory management is also undefined for the CPU implementation (C and ISPC backend), so it is fully dependent on the operating system. The program sometimes encounters an out-of-memory issue during the training process.

5 CONCLUSION

In this project, we implemented EGNN in Loma and conducted several experiments to evaluate its performance and effectiveness. We have learned how to incorporate the differentiable operators generated by the AD system into the PyTorch model. We designed our wrappers elegantly so that our Loma operators can be arbitrarily combined with PyTorch operators conveniently. It is also very fancy to use the PyTorch framework to train a neural network without any PyTorch arithmetic operator. We believe our project provides a good demonstration of the usage of the Loma Programming Language.

REFERENCES

- [1] Matthias Fey and Jan Eric Lenssen. 2019. Fast graph representation learning with PyTorch Geometric. *arXiv preprint arXiv:1903.02428* (2019).
- [2] Thomas N Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).
- [3] Victor Garcia Satorras, Emiel Hoogetboom, and Max Welling. 2021. E (n) equivariant graph neural networks. In *International conference on machine learning*. PMLR, 9323–9332.
- [4] Nathaniel Thomas, Tess Smidt, Steven Kearnes, Lusann Yang, Li Li, Kai Kohlhoff, and Patrick Riley. 2018. Tensor field networks: Rotation-and translation-equivariant neural networks for 3d point clouds. *arXiv preprint arXiv:1802.08219* (2018).